Research Note: LLD Practice Platform

1. The learner problem

LLD practice is easy to start but hard to judge. A learner can sketch classes for a Parking Lot in 40 minutes and still not know whether the responsibilities, abstractions and trade-offs hold up.

- **Many valid answers.** A pricing Strategy and a plain enum can both be right. Comparing against one reference solution marks a different-but-good design as wrong.
- **Good feedback is scarce.** The reliable signal is a senior engineer or mock interviewer: costly, and not available every day.
- **No memory across attempts.** Learners rarely notice they repeat the same mistake (god classes, missing state handling, tight coupling) across problems.

A learner needs a clear problem, a structured way to show the design, feedback that points at specific parts of what they wrote, and a history that shows which weaknesses keep coming back.

2. Existing approaches

Tool	How you practise	Submission	Feedback	Main limitation
Educative, *Grokking the LLD Interview* [3]	Read 16 case studies (Parking Lot, Chess, Library…)	None	None on your design; compare with the solution	Passive reading, no practice loop
awesome-low-level-design (GitHub, ~27k stars) [4]	Study reference implementations	None	Self-comparison	One reference reads as "the" answer
AlgoMaster LLD interviews [1]	AI mock over 45 problems: Requirements → Entities → Class design → Implementation	Chat + code editor	Score /10 per stage, judged with a reference solution; attempts saved	Scores stages, not design qualities; anchored to a reference
Codemia OOD [2]	70+ problems; draw UML class diagrams	Diagram	"AI Design Review", editorial solutions, progress dashboard	No published rubric
ChatGPT / Claude directly	Paste a design, ask "is this good?"	Free text	Unstructured opinion	Different answer each run, no history, often too agreeable

3. Key gaps

1. **Reference-anchored grading.** A valid design that differs from the model answer loses marks; the learner learns to copy, not to reason.
2. **Feedback without evidence.** "Improve cohesion" doesn't help unless it names the class and the line that caused it.
3. **One number, no direction.** 6/10 doesn't say which skill to work on next.
4. **No pattern across attempts.** No tool reviewed tells a learner "you missed state handling in 3 of your last 4 designs".
5. **Extensibility is never tested.** The real LLD skill is surviving a requirement change, yet practice ends at the first submission.

4. Product direction

Grade the design against the **requirements and a fixed rubric**, not a model answer, and make the learner **face a requirement change**.

Gap	MVP decision
Reference-anchored grading	Per-problem requirement checklist + fixed 6-dimension rubric (1–4 with anchor sentences). Any design meeting them scores well.
Feedback without evidence	Every finding must quote the submission; findings whose quote is not found in it are dropped by code.
One number, no direction	A score per rubric dimension plus one "fix this next".
No pattern across attempts	History shows the weakest dimension across all completed attempts.
Extensibility never tested	Attempt v2+ adds the problem's change request (e.g. "add EV charging billed per kWh"); feedback checks how the design absorbed it.

Submission format: structured text in five sections: requirements & assumptions, classes & responsibilities, key interfaces as code, main flow, trade-offs. It is the smallest format that shows both reasoning and abstraction. Diagrams add build effort but little extra evidence for an MVP, and the domain model keeps a slot for them.

Out of scope: diagram editor, running learner code, timers, peer review, courses.

Sources

1. AlgoMaster, LLD interview practice. https://algomaster.io/interview/low-level-design (accessed 13 Oct 2026)
2. Codemia, system design and OOD practice. https://www.codemia.io (accessed 13 Oct 2026)
3. Javarevisited, review of Educative's "Grokking the Low Level Design Interview". https://medium.com/javarevisited/review-is-grokking-the-low-level-design-interview-using-ood-principles-on-educative-worth-it-c49e3c65d7de (accessed Oct 2026)
4. awesome-low-level-design on HelloGitHub. https://hellogithub.com/en/repository/ashishps1/awesome-low-level-design (accessed 13 Oct 2026)